

元语言的严谨定义与罗素悖论的避免

摘要

本文完全专注于元语言本身，不使用比喻或类比，严格界定其构成、运作方式以及为何不会导致罗素悖论。全文依次给出元语言的五大模块定义（基本逻辑、自然数、集合概念、字符串与语法、语义概念），并从实践层面、原理层面和可严格化层面三重论证元语言免于悖论的根本原因。最后，以精炼的公式总结元语言的恰到好处的强与恰到好处的弱。

目录

1 引言	2
2 元语言的五大模块	2
2.1 元语言的基本逻辑	2
2.2 元语言的自然数	2
2.3 元语言的集合概念	3
2.3.1 朴素集合论概述（前置补充）	3
2.3.2 元语言中实际使用的集合操作（安全片段）	4
2.4 字符串与语法	5
2.5 语义概念	6
3 元语言如何避免罗素悖论	6
3.1 第一层：实际使用中从不构造“所有集合的集合”	6
3.2 第二层：分离公理模式的限制使用（核心屏障）	6
3.3 第三层：可严格化到无悖论的元理论	7
4 总结	8

1 引言

在为—阶逻辑提供语法和语义定义时，我们所使用的元语言（meta-language）是一种非形式化但精确描述的数学实践语言。它既不等于完全形式化的对象语言（如 ZFC 或—阶谓词演算），也不是模糊的日常语言。元语言是一种经过精密校准的工作语言，它的表达能力恰好足以定义—阶逻辑的全部组件，却不足以产生罗素悖论等自指悖论。

本文的使命是：正面给出元语言的严格定义，并在同一框架下证明其安全性。全文不使用比喻，不借助直觉类比，每一步都依赖明确的集合论操作边界。

2 元语言的五大模块

元语言至少包含以下五类初始概念和操作能力。每一类均以定义形式给出，边界明确。

2.1 元语言的基本逻辑

元语言内置经典二值逻辑的完整词汇。

定义 2.1 (命题连接词). 元语言包含以下命题连接词，其语义遵循通常的真值表：

且 ($\wedge$), 或 ($\vee$), 若...则... ($\rightarrow$), 当且仅当 ($\leftrightarrow$), 并非 ($\neg$).

定义 2.2 (量词). 元语言包含全称量词与存在量词：

对所有 x ($\forall x$), 存在 x ($\exists x$).

这些量词的变域涵盖元语言中所讨论的任意对象——自然数、符号、字符串、公式、集合等。量词不受类型限制，这是元语言区别于类型论的关键特征。

定义 2.3 (等词). 元语言包含等词 “=”，表示对象的同一性（identity），满足：

(i) 自反性： $\forall x (x = x)$;

(ii) 对称性： $\forall x \forall y (x = y \rightarrow y = x)$;

(iii) 传递性： $\forall x \forall y \forall z ((x = y \wedge y = z) \rightarrow x = z)$;

(iv) 可替换性：若 $x = y$ ，则在任何元语言表达式中可用 y 替换 x 而保持真值不变。

2.2 元语言的自然数

元语言预设一套关于自然数的非形式理论，其强度大致相当于原始递归算术（PRA）。

定义 2.4 (自然数的基本设定). 元语言接受：

(i) 一个起始对象 0（零）；

(ii) 一个后继运算 $'$ ：对每个自然数 n ， n' 是其唯一后继；

(iii) 归纳原理：若性质 P 满足 $P(0)$ 且 $\forall n (P(n) \rightarrow P(n'))$ ，则 $\forall n P(n)$ 。

定义 2.5 (原始递归定义). 元语言允许对自然数进行原始递归定义。例如, 加法 $+$ 与乘法 $\times$ 由以下递归方程定义:

$$\begin{aligned} n + 0 &= n, & n + m' &= (n + m)', \\ n \times 0 &= 0, & n \times m' &= (n \times m) + n. \end{aligned}$$

由此可进一步定义阶乘、指数、前驱等一切原始递归函数。

注记 2.1 (实无穷与潜无穷). 自然数全体 $\mathbb{N}$ 在标准数学元语言中被视为一个已完成的无穷集合 (实无穷立场)。若持构造主义立场, 则 $\mathbb{N}$ 仅作为一个生成过程 (潜无穷) 存在。本文采用标准的实无穷立场, 这是经典模型论通行的选择。

归纳原理与原始递归定义赋予元语言足够的表达力以编码语法对象 (如哥德尔编码), 这为第三层安全保障 (可严格化到 PRA) 提供了基础。

2.3 元语言的集合概念

这是元语言最核心也最易被误解的部分。元语言并不拥抱完整的朴素集合论, 而是仅使用其一个严格受限的安全片段。

2.3.1 朴素集合论概述 (前置补充)

在界定元语言的集合操作之前, 有必要先澄清朴素集合论本身的定义及其崩溃原因。

定义 2.6 (朴素集合论). 朴素集合论 (*naïve set theory*) 是由格奥尔格·康托尔 (*Georg Cantor*) 在 19 世纪创立的原始集合论。其核心原理是无限制概括公理 (*Unrestricted Comprehension Principle*):

对任意可表达的性质 (谓词) $P(x)$, 存在一个集合 S 恰好由所有满足 P 的对象构成, 记作

$$S = \{x \mid P(x)\}.$$

换言之, 任意性质决定一个集合。这一原理极其直观: 我们只需用语言描述一个汇集的条件, 该汇集便被承认为一个数学对象——集合。

例 2.1. 在朴素集合论中, 以下均被认可为集合:

$$\{x \mid x \text{ 是偶数}\}, \quad \{x \mid x \text{ 是集合}\}, \quad \{x \mid x \notin x\}.$$

第三个例子直接通向悖论。

定义 2.7 (罗素悖论 (Russell's Paradox, 1901)). 令 $R = \{x \mid x \notin x\}$ 。由无限制概括公理, R 是一个集合。现在考察 $R \in R$ 是否成立:

- 若 $R \in R$, 则根据 R 的定义, R 的元素必须满足 $x \notin x$, 故 $R \notin R$, 矛盾。
- 若 $R \notin R$, 则 R 满足条件 $x \notin x$, 由概括公理 $R \in R$, 矛盾。

因此, 朴素集合论在经典逻辑下是不一致的。

注记 2.2 (历史注记). 罗素于 1901 年发现此悖论, 随后独立地由策梅洛 (Zermelo) 发现。这一发现直接推动了公理化集合论 (ZFC、NBG 等) 的诞生, 其核心修复策略是用严格的公理控制集合的存在, 而非允许任意性质决定集合。

下面的交换图总结了从无限制概括公理到罗素悖论的逻辑链条——图中每一步都是必然推导：

$$\begin{array}{c}
 \text{无限制概括公理: } \forall P \exists S (x \in S \leftrightarrow P(x)) \\
 \Downarrow \\
 \text{取 } P(x) := (x \notin x) \\
 \Downarrow \\
 R = \{x \mid x \notin x\} \text{ 是一个集合} \\
 \Downarrow \\
 R \in R \longleftrightarrow R \notin R \\
 \Downarrow \\
 \text{矛盾!}
 \end{array}$$

2.3.2 元语言中实际使用的集合操作（安全片段）

元语言弃用了无限制概括公理，代之以以下经过严格限制的操作。每一项操作都明确标注其安全边界。

定义 2.8 (原子对象). 元语言中预设一些基本的原子对象 (*urelements*)，它们本身不是集合，但可作为集合的元素。典型的原子对象包括：

符号：“ v ”，“ 0 ”，“ c ”，“ f ”，“ $\forall$ ”，“ $\exists$ ”，“ $\rightarrow$ ”，“ $($ ”，“ $)$ ”，...

这些原子是互不相同的抽象对象。

定义 2.9 (有穷集合的枚举构造). 可以从已有对象通过枚举形成有穷集合：

$$\{a_1, a_2, \dots, a_k\}.$$

该集合的元素恰为所列出的对象，不包含任何额外元素。

定义 2.10 (归纳生成的集合). 元语言接受如下形式的归纳定义：一个集合 S 是满足下列闭包条件的最小集合：

- (i) 基础条款： S 包含某些指定的初始元素；
- (ii) 归纳条款： S 在若干生成规则下封闭——若 $e_1, \dots, e_n \in S$ ，则某个由它们按规则构成的复合对象也属于 S ；
- (iii) 极小性条款： 没有其他对象属于 S （即 S 是所有满足 (i)(ii) 的集合之交）。

典型的例子是克林闭包 Σ^* ：所有有限字符串的集合。

下面的交换图刻画了归纳集合的逐层构造过程：

$$S_0 \longrightarrow S_1 \longrightarrow S_2 \longrightarrow \dots \longrightarrow S = \bigcup_{n \in \mathbb{N}} S_n$$

其中 S_0 为基础元素集， S_{n+1} 为对 S_n 施加所有生成规则所得结果的并集。 S 的极小性保证没有多余元素。

定义 2.11 (子集与分离 (核心安全机制)). 对于已存在的集合 X , 允许根据元语言中可表达的明确性质 P 分离出子集:

$$\{x \in X \mid P(x)\}.$$

关键限制: 分离必须指定一个来源域 X . 元语言从不断言存在不指定来源域的集合 $\{x \mid P(x)\}$. 这是元语言区别于朴素集合论概括公理的核心安全机制, 也是阻断罗素悖论的直接屏障。

定义 2.12 (有序对与关系). 有序对 (a, b) 可视为原始概念, 也可通过库拉托夫斯基对 (*Kuratowski pair*) 在集合论内部定义为

$$(a, b) := \{\{a\}, \{a, b\}\}.$$

函数 $f: X \rightarrow Y$ 定义为满足条件 $\forall x \in X \exists! y \in Y ((x, y) \in f)$ 的二元关系 $f \subseteq X \times Y$. 进而可构造笛卡尔积 $X \times Y$ 与函数空间 Y^X 等。

定义 2.13 (有限序列与克林闭包). 对于已存在的集合 X , X 中元素的有限序列的全体称为 X 的克林闭包 (*Kleene closure*), 记作 X^* , 其归纳定义如下:

(i) 空序列 $\varepsilon \in X^*$;

(ii) 若 $s \in X^*$ 且 $x \in X$, 则将 x 添加到 s 末尾所得的序列也属于 X^* ;

(iii) 仅由 (i)(ii) 生成的序列属于 X^* 。

定义 2.14 (不预设全集). 元语言中从不断言存在一个包含一切集合或一切对象的全集 (*universal set*). 集合总是从某些已给定的原子或集合出发, 通过上述受限制的操作逐步构建而成。这从根本上切断了 $\{x \mid x = x\}$ 之类的大全集构造。

2.4 字符串与语法

在拥有了集合操作能力之后, 一阶逻辑的语法对象可以被严格定义。

定义 2.15 (符号). 符号是元语言中的原子对象。全体符号 (字母表) Σ 是一个有限或可数的集合, 且其中任意两个符号互不相同。

定义 2.16 (字符串). Σ 中元素的有限序列称为 Σ 上的字符串。全体字符串的集合为克林闭包 Σ^* , 由上述归纳生成规则保证其存在。

定义 2.17 (项与公式 (作为 Σ^* 的子集)). 项集 Term 和公式集 Form 是 Σ^* 中满足特定归纳闭包条件的子集。形式化地:

$$\text{Term} = \{s \in \Sigma^* \mid s \text{ 满足项的归纳闭包条件}\},$$

$$\text{Form} = \{s \in \Sigma^* \mid s \text{ 满足公式的归纳闭包条件}\}.$$

这是对已存在的集合 Σ^* 应用分离公理的实例。其“最小性”条件可以用自然数归纳法严格表述: $s \in \text{Term}$ 当且仅当存在某个自然数 n 和一个长度为 n 的生成序列证明 s 可由基础元素经有限次应用生成规则得到。

下面的交换图展示了从符号出发的完整构造层次:

$$\Sigma \xrightarrow{\text{克林闭包}} \Sigma^* \xrightarrow{\text{归纳分离}} \text{Term} \xrightarrow{\text{归纳分离}} \text{Form}$$

2.5 语义概念

语义赋予语法对象以数学意义。

定义 2.18 (结构). 一个结构 $\mathfrak{A}$ 由元语言中的一个非空集合 A (论域) 和一组函数、关系组成。这些函数和关系都是元语言中的合法对象。

定义 2.19 (赋值). 赋值是元语言中的函数，其定义域为变量集合，值域为论域 A 。

定义 2.20 (满足关系). 满足关系 $\models$ 通过元语言中对公式复杂度的归纳定义给出。在归纳步骤中，元语言量词“对所有 $a \in A$ ”出现在全称量词语义条款中——这是合法的元语言量词使用，因为 A 是一个已存在的确定集合。

至此，一阶逻辑的完整语法与语义均在元语言内被定义完毕。

3 元语言如何避免罗素悖论

罗素悖论出现于无限制的概括公理模式：由任何性质 $\{x \mid P(x)\}$ 都可形成一个集合，尤其当取 $P(x)$ 为 $x \notin x$ 时导致矛盾。

元语言不会导出这一悖论，原因有三层，层次由浅入深。

3.1 第一层：实际使用中从不构造“所有集合的集合”

在定义一阶逻辑时，我们需要构造的集合极其具体且范围明确：

- 符号的集合 Σ ——由我们显式列出的有限或可数无穷集；
- Σ^* ——所有有限字符串的集合，通过归纳定义保证存在；
- 项集 Term 和公式集 Form——作为 Σ^* 的特定子集，通过分离获得；
- 对于给定的结构 $\mathfrak{A}$ ，只考虑论域 A 及其上的幂集、积集等，这些都是从 A 出发用分离和配对等受控操作得到的具体集合。

没有任何步骤需要构造一个“包含一切集合的集合”或“所有序数的集合”这样的全域对象。我们所讨论的集合总是限制在一个特定的、有明确生成基础的区域内——本质上是良基集合宇宙中由原子符号生成的一个极小片段。

注记 3.1. 这与 ZFC 集合论中通过累积层级 V_α 构造集合宇宙的精神一脉相承：集合总是从底层逐步生成，不存在自指的全域。

3.2 第二层：分离公理模式的限制使用（核心屏障）

元语言中使用的朴素集合操作，其本质全部是受限制的概括 (restricted comprehension)：

元语言从不单独写 $\{x \mid P(x)\}$ 而不指定其来源域。

每次形成集合，都限定在已事先存在的集合 X 内做分离，即形式为

$$\{x \in X \mid P(x)\}.$$

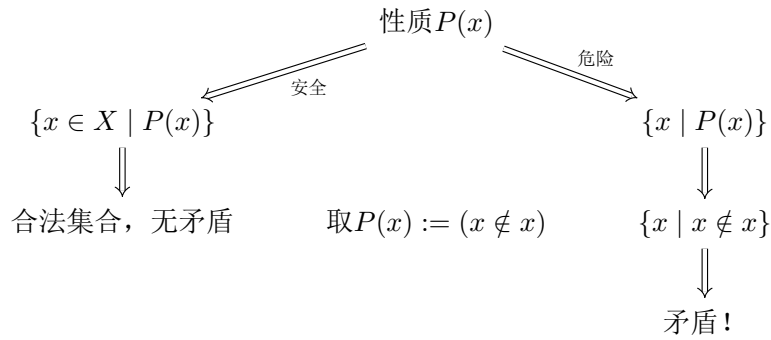
这直接阻断了罗素悖论的推理链条。因为罗素悖论的致命一步恰是构造“包含所有不包含自身的集合的那个集合” $R = \{x \mid x \notin x\}$ ——这个构造缺少来源域限制，因而在元语言中不被允许。

以公式集 Form 的定义为例：

$$\text{Form} = \{s \in \Sigma^* \mid s \text{ 满足公式的归纳闭包条件}\}.$$

这本质上是在 Σ^* 上应用分离，而非凭空由性质生成集合。

下面的交换图鲜明对比了安全路径与危险路径的分岔点：

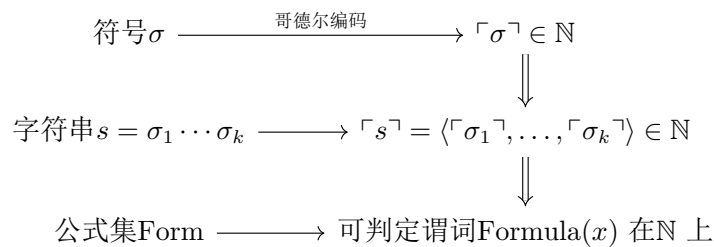


3.3 第三层：可严格化到无悖论的元理论

如果希望完全消除对朴素集合的任何不安，可以将上述元语言严格化到以下两类已被证明一致的系统中：

- (1) **原始递归算术 (PRA)**：在 PRA 中，根本没有“集合”这一对象类型。取而代之的是自然数和定义在自然数上的谓词，通过哥德尔编码将符号、字符串、公式等语法对象一一编码为自然数。归纳定义和归纳证明均可归约为对自然数的原始递归定义和归纳。
- (2) **有限类型论 (finite type theory)**：使用严格分层的类型结构——个体属于类型 0，个体集合属于类型 1，个体集合的集合属于类型 2，以此类推。此时 $x \in y$ 仅在 y 的类型比 x 高一级时才是合式公式。从而 $x \in x$ 根本不被语法所允许，罗素悖论在语法层面即被消解。

下面的交换图以 PRA 方案为例，展示语法对象如何编码为自然数：



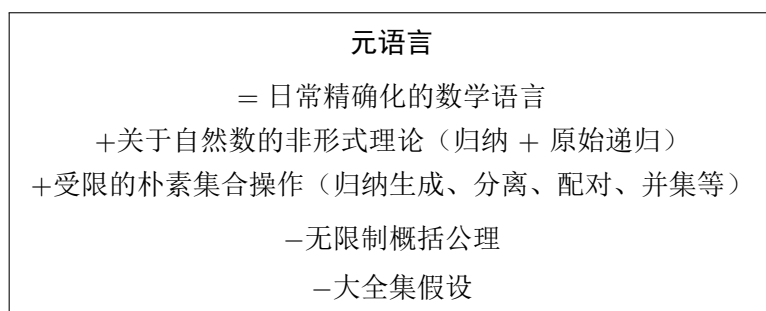
无论是 PRA 还是有限类型论，其一致性都已在极弱的元理论中被证明（PRA 的一致性证明甚至在命题逻辑加无量词归纳的系统中即可完成）。因此可以确信：

标准数学实践中使用的元语言，可以在不引入悖论的前提下被完全形式化。

在讨论一阶逻辑的元理论时所需的集合论操作——归纳生成集合、在已有集合上分离、配对、并集等——都可以在 ZFC 去掉无穷公理甚至更弱的系统中完成，而该弱系统的相对一致性已被确立。

4 总结

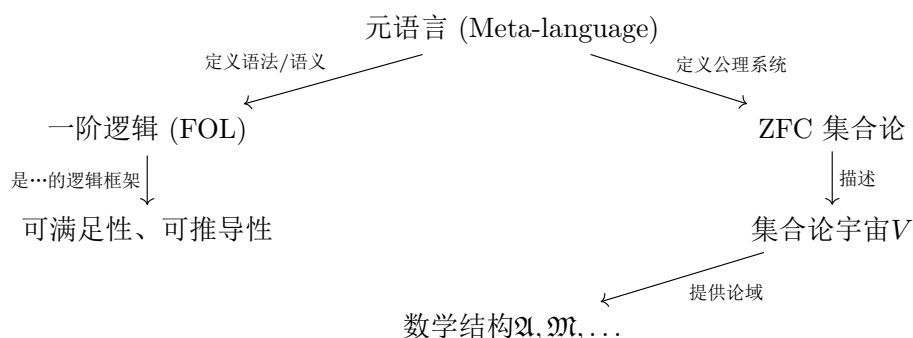
经过上述三个层面的分析，可以给出元语言的精确特征画像：



元语言之所以免于罗素悖论，不是因为它拥有某种形式化的防护措施，而是因为它的实践用法永远停留在构造具体、有底的语法和语义对象的范围内，从不涉及自指的全域构造。

注记 4.1 (对象语言与元语言的层次分离). 一旦我们需要研究“集合”这个概念本身（如集合的公理化、基数的性质、连续统假设等），我们便跳入对象语言（如 ZFC 公理化集合论），在其中用严格的公理控制集合的存在。此时，原来的元语言退居幕后，成为研究该形式系统的工具语言。这种严格的层次分离天然避免了语义悖论。

下面的交换图刻画了元语言、一阶逻辑和集合论之间的层次结构：



该图揭示了一个根本性的智慧分层：元语言同时定义了一阶逻辑的语法/语义和 ZFC 的公理系统。一阶逻辑作为逻辑框架被 ZFC 采用（ZFC 是一阶理论）；而 ZFC 所描述的集合论宇宙 V 又为研究数学结构提供了丰富的论域。元语言始终处于这一层次结构的顶端，它的任务是指定规则，而非成为自身规则的应用对象。

总之：

